PS11 : Telemedicine Slot Booking & Prescription Portal

NAME : DAKSHA SG
SRN:PES1UG24CS137

Justification :

Architecture Selection

We chose Layered Architecture for the Telemedicine Slot Booking & Prescription Portal. The system is organised into three layers: Presentation Layer, Business Layer, and Data Layer.
Presentation Layer : Patient Portal UI
Business Layer : Booking Manager, Payment Service, and Prescription Service components
Data Layer : Database component.

Reasons for Choosing Layered Architecture

1. Clear separation of responsibilities:
Layered Architecture separates the user interface, business operations, and data storage into different layers. In our telemedicine system, the Patient Portal handles user interaction, while the Booking Manager, Payment Service, and Prescription Service handle the core business operations. This makes the system easier to understand, develop, test, and maintain.

2. Suitable for the different functions of the system:
The system performs different operations such as booking consultation slots, processing payments, and generating prescriptions. Keeping these operations in separate business components allows each function to communicate through well-defined interfaces such as the Booking API, Payment Processing API, and Prescription API. This makes changes to one part of the system less likely to affect other layers.

Security Advantage

Layered Architecture provides separation between the user interface, business logic, and data layer, which helps restrict direct access to sensitive patient and prescription data. For example, the Patient Portal communicates with business services through defined interfaces rather than directly accessing the database. This provides controlled access to sensitive healthcare information.

Performance Benefit

The separation of responsibilities allows the system to handle different operations independently within their respective components. For example, booking and prescription-related operations can be handled by their corresponding business components while data is accessed through the database layer. This can reduce unnecessary processing between unrelated parts of the system and improve the overall responsiveness of the application.